

ДПО	Фронтенд и бэкенд разработка
ДИСЦИПЛИНА	Основы фронтенд-разработки
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания к практическим занятиям

Практическое занятие 13: Создание простого приложения на React

Цель занятия

Научиться создавать React-приложение с нуля, освоить основные концепции: компоненты, пропсы, состояние, обработку событий и условный рендеринг. В результате будет создано работающее приложение - список задач с возможностью добавления, отметки выполнения и удаления. Также рассмотрим процесс публикации React-приложения на GitHub Pages.

Теоретическая справка

Как создается React-приложение

Современный способ создания React-приложений - использование инструмента Vite. Vite - это современный инструмент сборки, который используется вместо устаревшего Create React App. Он создает готовую структуру проекта с минимальной конфигурацией и обеспечивает быструю работу в режиме разработки.

Команда для создания проекта:

```
npm create vite@latest todo-app -- --template react
```

После создания нужно перейти в папку проекта, установить зависимости и запустить сервер разработки:

```
cd todo-app
npm install
npm run dev
```

Структура проекта

Созданный проект имеет следующую структуру ключевых файлов:

- `src/main.jsx` - точка входа, здесь React монтируется в DOM
- `src/App.jsx` - корневой компонент приложения
- `src/App.css` - стили для корневого компонента
- `index.html` - HTML-страница с контейнером `<div id="root">`

Основные понятия React

Компонент - это функция, которая возвращает JSX-разметку. Каждый компонент отвечает за свою часть интерфейса и может быть переиспользован. Любое React-приложение строится из компонентов.

Компоненты могут быть вложенными и переиспользованными. Их именование начинается с заглавной буквы. Компонент получает пропсы (входные данные) и может иметь состояние.

```
function Greeting() {  
  return <h1>Привет, мир!</h1>;  
}
```

JSX - синтаксис, позволяющий писать HTML-подобную разметку внутри JavaScript. В JSX можно встраивать JavaScript-выражения в фигурных скобках.

```
const name = "Анна";  
return <p>Привет, {name}!</p>;
```

Пропсы (props) - входные данные, которые родительский компонент передает дочернему. Дочерний компонент не может изменять пропсы — они доступны только для чтения.

```
function Welcome({ name }) {  
  return <h1>Привет, {name}!</h1>;  
}  
  
// Использование:  
<Welcome name="Иван" />
```

Состояние (state) - данные, которые компонент хранит и может изменять. При изменении состояния компонент перерисовывается автоматически. Состояние создается с помощью хука `useState`.

Пример добавления состояния в функциональный компонент с помощью хука `useState`.

```
const [state, setState] = useState(initialValue);
```

При использовании `useState` изменение состояния всегда происходит через функцию `setState` (работает асинхронно). Прямое изменение `state` не вызывает перерендер. Для зависимых обновлений используется функциональная форма: `setCount(prev => prev + 1)`

```
import { useState } from 'react';  
  
function Counter() {  
  const [count, setCount] = useState(0);
```

```
return (  
  <div>  
    <p>Счёт: {count}</p>  
    <button onClick={() => setCount(count + 1)}>  
      Увеличить  
    </button>  
  </div>  
>);  
>
```

Обработка событий в React похожа на обработку в обычном JavaScript, но с особенностями: имена событий пишутся в camelCase, а в обработчик передается функция.

```
<button onClick={handleClick}>Нажми меня</button>  
<input onChange={(e) => console.log(e.target.value)} />  
<form onSubmit={handleSubmit}>...</form>
```

Условный рендеринг позволяет показывать разные части интерфейса в зависимости от состояния.

```
// Вариант с && (показать если условие истинно)  
{isLoading && <p>Загрузка...</p>}  
  
// Вариант с тернарным оператором (выбор из двух)  
{isLoggedIn ? <LogoutButton /> : <LoginButton />}
```

Рендеринг списков выполняется с помощью метода map, при этом каждый элемент должен получать уникальный атрибут key.

```
const items = ['яблоко', 'банан', 'апельсин'];  
return (  
  <ul>  
    {items.map((item, index) => (  
      <li key={index}>{item}</li>  
    ))}  
  </ul>  
>);
```

Разбор примеров

Пример 1: Счётчик с ограничением

Создадим компонент счётчика, который нельзя увеличить больше 10 и нельзя уменьшить меньше 0.

```
import { useState } from 'react';

function LimitedCounter() {
  const [count, setCount] = useState(0);

  const increment = () => {
    if (count < 10) {
      setCount(count + 1);
    }
  };

  const decrement = () => {
    if (count > 0) {
      setCount(count - 1);
    }
  };

  return (
    <div style={{ textAlign: 'center', padding: '20px' }}>
      <h2>Счётчик: {count}</h2>
      <button onClick={decrement} disabled={count === 0}>-</button>
      <button onClick={increment} disabled={count === 10}>+</button>
      {count === 10 && <p style={{color: 'red'}}>Достигнут максимум!</p>}
      {count === 0 && <p style={{color: 'gray'}}>Минимальное значение</p>}
    </div>
  );
}
```

Как это работает:

- `useState(0)` создает состояние с начальным значением 0.
- Проверки `if (count < 10)` и `if (count > 0)` предотвращают выход за границы.
- Атрибут `disabled` блокирует кнопки, когда достигнуты пределы.
- Условный рендеринг показывает подсказки при достижении границ.

Пример 2: Приветствие с именем

Компонент, который приветствует пользователя по имени, введенному в поле ввода.

```
import { useState } from 'react';

function NameGreeting() {
  const [name, setName] = useState('');
  const [greeting, setGreeting] = useState('');

  const handleInputChange = (e) => {
    setName(e.target.value);
  };

  const sayHello = () => {
```

```

    if (name.trim()) {
      setGreeting(`Привет, ${name}!`);
    } else {
      setGreeting('Пожалуйста, введите имя');
    }
  };

  return (
    <div style={{ padding: '20px' }}>
      <h2>Приветствие</h2>
      <input
        type="text"
        value={name}
        onChange={handleInputChange}
        placeholder="Введите ваше имя"
      />
      <button onClick={sayHello}>Поздороваться</button>
      {greeting && <p>{greeting}</p>}
    </div>
  );
}

```

Как это работает:

- Состояние `name` синхронизируется со значением поля ввода.
- Состояние `greeting` хранит текст приветствия.
- Проверка `if (name.trim())` отсекает пустые строки.
- `{greeting && <p>{greeting}</p>}` показывает сообщение только когда оно есть.

Пример 3: Переключатель темы

Компонент, который переключает светлую и темную тему приложения.

```

import { useState } from 'react';

function ThemeSwitcher() {
  const [isDark, setIsDark] = useState(false);

  const toggleTheme = () => {
    setIsDark(!isDark);
  };

  const themeStyles = {
    backgroundColor: isDark ? '#333' : '#fff',
    color: isDark ? '#fff' : '#333',
    minHeight: '200px',
    padding: '20px',
    transition: 'all 0.3s'
  };

  return (

```

```
<div style={themeStyles}>
  <h2>{isDark ? 'Тёмная тема' : 'Светлая тема'}</h2>
  <button onClick={toggleTheme}>
    Переключить на {isDark ? 'светлую' : 'тёмную'}
  </button>
</div>
);
}
```

Как это работает:

- Состояние `isDark` хранит текущую тему.
- Объект `themeStyles` динамически формируется на основе состояния.
- Тернарные операторы подбирают нужные цвета и текст.

Постановка задачи: Менеджер задач

Нам предстоит реализовать приложение для управления задачами со следующим функционалом:

- Добавление новых задач через форму ввода.
- Отметка задач как выполненных.
- Удаление задач.
- Фильтрация задач: все, активные, выполненные.
- Счетчик оставшихся задач.
- Сохранение задач в `localStorage` (чтобы после обновления страницы они не пропадали).

Пошаговое руководство

Шаг 1: Создание проекта

1. Откройте терминал и выполните команду:

```
npm create vite@latest todo-manager -- --template react
```

2. Перейдите в папку проекта:

```
cd todo-manager
```

3. Установите зависимости:

```
npm install
```

Шаги выше мы уже выполнили в начале практики, поэтому, перейдем далее

- Удалите лишние файлы. В папке `src` оставьте только `main.jsx`, удалите `App.css` и `index.css` (стили будем писать в компонентах).
- Очистите `App.jsx` до базовой структуры:

```
function App() {  
  return (  
    <div>  
      <h1>Менеджер задач</h1>  
    </div>  
  );  
}  
  
export default App;
```

- Запустите сервер разработки:

```
npm run dev
```

Шаг 2: Создание компонента задачи (TodoItem)

Создайте в папке `src` папку `components`, а в ней файл `TodoItem.jsx`:

```
function TodoItem({ task, onToggle, onDelete }) {  
  return (  
    <li style={{  
      display: 'flex',  
      alignItems: 'center',  
      gap: '10px',  
      padding: '8px',  
      borderBottom: '1px solid #eee'  
    }}>  
      <input  
        type="checkbox"  
        checked={task.completed}  
        onChange={() => onToggle(task.id)}  
      />  
      <span style={{  
        flex: 1,  
        textDecoration: task.completed ? 'line-through' : 'none',  
        color: task.completed ? '#999' : '#333'  
      }}>  
        {task.text}  
      </span>  
      <button  
        onClick={() => onDelete(task.id)}  
        style={{  
          background: '#ff4444',  

```

```

        color: 'white',
        border: 'none',
        borderRadius: '4px',
        padding: '4px 8px',
        cursor: 'pointer'
      }}
    >
      Удалить
    </button>
  </li>
);
}

export default TodoItem;

```

Что здесь происходит:

- Компонент принимает три пропса: `task` (объект задачи), `onToggle` (функция для отметки выполнения), `onDelete` (функция для удаления).
- Чекбокс привязан к состоянию `task.completed`.
- Текст задачи зачеркивается, если задача выполнена.
- Кнопка удаления вызывает `onDelete` с идентификатором задачи.

Шаг 3: Создание формы добавления (AddTodoForm)

Создайте файл `AddTodoForm.jsx` в папке `components`:

```

import { useState } from 'react';

function AddTodoForm({ onAdd }) {
  const [text, setText] = useState('');

  const handleSubmit = (e) => {
    e.preventDefault();
    if (text.trim()) {
      onAdd(text.trim());
      setText('');
    }
  };

  return (
    <form onSubmit={handleSubmit} style={{ marginBottom: '20px' }}>
      <input
        type="text"
        value={text}
        onChange={(e) => setText(e.target.value)}
        placeholder="Добавить новую задачу..."
        style={{
          padding: '8px',
          width: '300px',
          marginRight: '10px',

```

```

        borderRadius: '4px',
        border: '1px solid #ddd'
      }}
    />
    <button
      type="submit"
      style={{
        padding: '8px 16px',
        background: '#4CAF50',
        color: 'white',
        border: 'none',
        borderRadius: '4px',
        cursor: 'pointer'
      }}
    >
      Добавить
    </button>
  </form>
);
}

export default AddTodoForm;

```

Что здесь происходит:

- Локальное состояние `text` управляет значением поля ввода.
- При отправке формы вызывается `preventDefault` для предотвращения перезагрузки.
- Если текст не пустой, вызывается переданная функция `onAdd`, а поле очищается.

Шаг 4: Создание фильтров (TodoFilters)

Создайте файл `TodoFilters.jsx`:

```

function TodoFilters({ filter, onFilterChange, activeCount }) {
  return (
    <div style={{
      display: 'flex',
      justifyContent: 'space-between',
      alignItems: 'center',
      marginBottom: '20px',
      paddingBottom: '10px',
      borderBottom: '2px solid #eee'
    }}>
      <span>Осталось задач: {activeCount}</span>
    </div>
    {[ 'all', 'active', 'completed' ].map((filterType) => (
      <button
        key={filterType}
        onClick={() => onFilterChange(filterType)}
        style={{
          margin: '0 5px',

```

```

        padding: '5px 10px',
        background: filter === filterType ? '#007bff' : '#f0f0f0',
        color: filter === filterType ? 'white' : '#333',
        border: 'none',
        borderRadius: '4px',
        cursor: 'pointer'
      }}
    >
    {filterType === 'all' ? 'Все' :
      filterType === 'active' ? 'Активные' : 'Выполненные'}
    </button>
  )})
</div>
</div>
);
}

export default TodoFilters;

```

Что здесь происходит:

- Компонент принимает текущий фильтр и функцию его изменения.
- Кнопка активного фильтра выделяется синим цветом.
- Счетчик показывает количество невыполненных задач.

Шаг 5: Сборка главного компонента (App.jsx)

Теперь соберем всё вместе в `App.jsx`:

```

import { useState, useEffect } from 'react';
import AddTodoForm from './components/AddTodoForm';
import TodoFilters from './components/TodoFilters';
import TodoItem from './components/TodoItem';

function App() {
  // Состояние для списка задач
  const [todos, setTodos] = useState(() => {
    // Загружаем сохраненные задачи из localStorage
    const saved = localStorage.getItem('todos');
    return saved ? JSON.parse(saved) : [];
  });

  // Состояние для текущего фильтра
  const [filter, setFilter] = useState('all');

  // Сохраняем задачи в localStorage при каждом изменении
  useEffect(() => {
    localStorage.setItem('todos', JSON.stringify(todos));
  }, [todos]);

  // Добавление новой задачи

```

```
const addTodo = (text) => {
  const newTodo = {
    id: Date.now(),
    text: text,
    completed: false
  };
  setTodos([...todos, newTodo]);
};

// Переключение статуса задачи
const toggleTodo = (id) => {
  setTodos(todos.map(todo =>
    todo.id === id ? { ...todo, completed: !todo.completed } : todo
  ));
};

// Удаление задачи
const deleteTodo = (id) => {
  setTodos(todos.filter(todo => todo.id !== id));
};

// Фильтрация задач
const filteredTodos = todos.filter(todo => {
  if (filter === 'active') return !todo.completed;
  if (filter === 'completed') return todo.completed;
  return true; // 'all'
});

// Подсчет активных задач
const activeCount = todos.filter(todo => !todo.completed).length;

return (
  <div style={{
    maxWidth: '600px',
    margin: '0 auto',
    padding: '20px',
    fontFamily: 'Arial, sans-serif'
  }}>
    <h1 style={{ textAlign: 'center', color: '#333' }}>Менеджер задач</h1>

    <AddTodoForm onAdd={addTodo} />

    <TodoFilters
      filter={filter}
      onFilterChange={setFilter}
      activeCount={activeCount}
    />

    {filteredTodos.length === 0 ? (
      <p style={{ textAlign: 'center', color: '#999' }}>
        {filter === 'all' ? 'Задач пока нет' :
        filter === 'active' ? 'Нет активных задач' : 'Нет выполненных задач'}
      </p>
    ) : (
```

```

    <ul style={{ listStyle: 'none', padding: 0 }}>
      {filteredTodos.map(todo => (
        <TodoItem
          key={todo.id}
          task={todo}
          onToggle={toggleTodo}
          onDelete={deleteTodo}
        />
      ))}
    </ul>
  )}

  {todos.length > 0 && (
    <button
      onClick={() => setTodos([])}
      style={{
        marginTop: '20px',
        padding: '8px 16px',
        background: '#dc3545',
        color: 'white',
        border: 'none',
        borderRadius: '4px',
        cursor: 'pointer',
        width: '100%'
      }}
    >
      Очистить всё
    </button>
  )}
</div>
);
}

export default App;

```

Ключевые моменты в App.jsx:

- `useState` с функцией инициализации для загрузки из `localStorage`.
- `useEffect` для сохранения задач при каждом изменении.
- Функции `addTodo`, `toggleTodo`, `deleteTodo` для управления задачами.
- Фильтрация задач в зависимости от выбранного фильтра.
- Условный рендеринг для пустого состояния.
- Передача пропсов дочерним компонентам.

Шаг 6: Точка входа (main.jsx)

Убедитесь, что `main.jsx` выглядит так:

```

import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';

```

```
ReactDOM.createRoot(document.getElementById('root')).render(  
  <React.StrictMode>  
    <App />  
  </React.StrictMode>  
);
```

Дополнительные задания для самостоятельной работы

После создания базового приложения можете добавить следующие улучшения:

1. **Редактирование задачи** — при двойном клике на текст задачи должно появляться поле для редактирования.
2. **Смена темы** — реализуйте переключение между светлой и темной темой.

Публикация на GitHub Pages

Шаг 1: Подготовка репозитория

1. Создайте новый репозиторий на GitHub.
2. Инициализируйте git в локальном проекте и свяжите с репозиторием.

Шаг 2: Настройка Vite для GitHub Pages

Установите пакет `gh-pages`:

```
npm install --save-dev gh-pages
```

В файле `package.json` добавьте:

```
"homepage": "https://ваш-логин.github.io/todo-app" // тут полный путь до Git Pages
```

В раздел `scripts` добавьте:

```
"predeploy": "npm run build",  
"deploy": "gh-pages -d dist"
```

Шаг 3: Настройка конфигурации Vite

Создайте или отредактируйте файл `vite.config.js`:

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  base: '/todo-app/' // тут название репозитория
})
```

Шаг 4: Деплой

Выполните команду:

```
npm run deploy
```

Шаг 5: Настройка GitHub Pages

1. Перейдите в настройки репозитория на GitHub (вкладка Settings).
2. В разделе Pages выберите ветку `gh-pages` и папку `/root` <- **ВАЖНО!**
3. Сохраните — через несколько минут приложение будет доступно по адресу: <https://ваш-логин.github.io/todo-app>.

Что делать при обновлении приложения

При внесении изменений просто выполните:

```
npm run deploy
```

Изменения автоматически соберутся и загрузятся на GitHub Pages.

Формат отчёта

В область для загрузки (Практическое задание 13) прикреплена ссылка на GitHub Pages. Страница React-приложения проверена и работоспособна.